

SKILL – 3

Code(Command History & Input Buffer):

```
GNU nano 8.7.1 Skill-03.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 20
#define MAX_HISTORY 10

int main() {
    char **history = malloc(MAX_HISTORY * sizeof(char *));
    int history_count = 0;

    char *input = malloc(INITIAL_SIZE);
    int buffer_size = INITIAL_SIZE;

    if (history == NULL || input == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("==== COMMAND HISTORY DEMO =====\n");
    printf("Escape sequence example: \\033[A = UP arrow\n");
    printf("Escape sequence example: \\033[B = DOWN arrow\n");

    /* Store commands */
    const char *commands[] = {
        "ls",
```

```

        "pwd",
        "gcc program.c",
        "git status",
        "git push"
    };

    for (int i = 0; i < 5; i++) {
        history[history_count] = malloc(strlen(commands[i]) + 1);

        if (history[history_count] == NULL) {
            printf("Memory allocation failed\n");
            break;
        }

        strcpy(history[history_count], commands[i]);
        history_count++;
    }

    /* Display command history */
    printf("\nCommand History:\n");

    for (int i = 0; i < history_count; i++) {
        printf("%d %s\n", i + 1, history[i]);
    }

    /* Free allocated memory */
    for (int i = 0; i < history_count; i++) {
        free(history[i]);
    }

    free(history);
    free(input);

    return 0;
}

```

Output:

```

uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-03$ nano Skill-03.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-03$ gcc Skill-03.c -o Skill-03
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-03$ ./Skill-03
===== COMMAND HISTORY DEMO =====
Escape sequence example: \033[A = UP arrow
Escape sequence example: \033[B = DOWN arrow

Command History:
1  ls
2  pwd
3  gcc program.c
4  git status
5  git push

```

Part – B:

Code(Dynamic Buffer + Linked List):

```
GNU nano 8.7.1 Dynamic.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Linked list node */
struct Node {
    int data;
    struct Node *next;
};

int main() {

    printf("==== DYNAMIC MEMORY AND LINKED LIST DEMO =====\n\n");

    /* Dynamic array */
    int capacity = 3;
    int size = 0;

    int *buffer = malloc(capacity * sizeof(int));

    if (buffer == NULL) {
        printf("Initial memory allocation failed.\n");
        return 1;
    }

    printf("Initial buffer capacity: %d\n", capacity);

    /* Add elements */
    for (int i = 0; i < 6; i++) {

        /* Resize before buffer becomes full */
        if (size == capacity) {
            capacity *= 2;

            int *temp = realloc(buffer,
                                capacity * sizeof(int));
        }
    }
}
```

Output:

```
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-03$ nano Dynamic.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-03$ gcc Dynamic.c -o Dynamic
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-03$ ./Dynamic
==== DYNAMIC MEMORY AND LINKED LIST DEMO =====

Initial buffer capacity: 3
Buffer resized. New capacity: 6

Dynamic Array Elements:
10 20 30 40 50 60

Creating linked list...

Linked List:
10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL

All dynamically allocated memory freed.
Program completed successfully.
```

Check for the Valgrind :

```
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-03$ valgrind --leak-check=full ./skill3_dynamic_memory
==3583== Memcheck, a memory error detector
==3583== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==3583== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==3583== Command: ./skill3_dynamic_memory
==3583==
===== DYNAMIC MEMORY AND LINKED LIST DEMO =====

Initial buffer capacity: 3
Buffer resized. New capacity: 6

Dynamic Array Elements:
10 20 30 40 50 60

Creating linked list...

Linked List:
10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL

All dynamically allocated memory freed.
Program completed successfully.
==3583==
==3583== HEAP SUMMARY:
==3583==    in use at exit: 0 bytes in 0 blocks
==3583==   total heap usage: 9 allocs, 9 frees, 1,156 bytes allocated
==3583==
==3583== All heap blocks were freed -- no leaks are possible
==3583==
==3583== For lists of detected and suppressed errors, rerun with: -s
==3583== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```